Claude Certified Developer – Foundations (CCDV-F)

Kompakt-Spickzettel für die Prüfung · Exam Guide v1.0 (gültig ab Juli 2026) · 8 Domains · 25 Skills

53 Items · 120 Min · Bestehen ab 720/1000
≈ 2:15 Min pro Frage · jede Frage beantworten (kein Malus)

Prüfung auf einen Blick

Format	Multiple Choice + **Multiple Response** (Anzahl wird genannt – genau so viele wählen, kein Teilpunkt)
Bewertung	Scaled Score 100–1.000, **Pass 720**, kriterienbasiert (nicht gegen andere Kandidaten). Domain-% nur diagnostisch.
Ablauf	Anthropic Partner Academy → Pearson VUE (online oder Testcenter). Umbuchen bis **24 h** vorher. NDA vor Start.
Gebühr / Gültigkeit	**$125**/Versuch · gültig **12 Monate** · rechtzeitige Verlängerung kostenlos (unbeaufsichtigtes Assessment)
Wiederholung	Wartezeit **14 / 30 / 90 Tage** · max. **4 Versuche** pro rollierende 12 Monate
Ausweis	Gültiger amtlicher Lichtbildausweis, Name exakt wie bei Registrierung

Gewichtung der Domains (%)

D1 | D2 33,1 | D5 | D6 | D7 | D8

Domain	%	Skills (Gewicht %)
D1 Agents & Workflows	14,7	1.1 Architektur 4,5 · 1.2 Konstruktion 5,3 · 1.3 Patterns 4,9
D2 Applications & Integration	**33,1**	2.1 Requirements 3,4 · 2.2 Life Cycle 2,8 · **2.3 API 6,8** · **2.4 SWE 7,4** · **2.5 App Design 8,6** · 2.6 Config 4,1
D3 Claude Code	3,1	3.1 Bedienung 3,1
D4 Eval, Test, Debug	2,6	4.1 Debugging & Errors 2,6
D5 Model Selection	16,8	5.1 LLM-Grundlagen 5,2 · **5.2 Technik 6,1** · 5.3 Modellwahl 2,7 · 5.4 Kosten 2,8
D6 Prompt & Context	11,0	6.1 Context 3,8 · 6.2 Prompt 4,6 · 6.3 Output 2,6
D7 Security & Safety	8,1	7.1 Security 3,2 · 7.2 Guardrails 2,3 · 7.3 Hooks 1,0 · 7.4 Secrets 1,6
D8 Tools & MCPs	10,6	8.1 Tools 4,4 · 8.2 MCP-Server 2,1 · 8.3 Customization 4,1

Die 10 Antwort-Heuristiken

1. **Einfachste ausreichende Lösung** gewinnt: Single Call → Workflow → Agent → Multi-Agent.
2. **„Muss immer / darf nie"** → deterministische Kontrolle (Code-Gate, Hook, Permission-Regel, Schema, Sandbox) – **nie nur Prompt**.
3. **Untrusted Content** (Web, E-Mail, Docs, Tool-Results) = **Daten, keine Anweisungen** → abgegrenzt im User-Turn, nie im System-Prompt; Least Privilege; Approvals.
4. **Autorisierung** gehört in Code/Datenschicht, nie in den Prompt.
5. **Erst messen**: Erfolgskriterien + Evals vor/nach jeder Änderung; immer nur eine Sache ändern.
6. **Kosten**: erst Gratis-Hebel (Caching, Batch, Trimmen, Output kürzen), dann Trade-offs (Effort, kleineres Modell).
7. **Wiederverwendbare Integration** über Apps → MCP-Server · **App-spezifische Aktion** → Custom Tool · **Know-how/Prozedur** → Skill · **generische Fähigkeit** → Built-in Tool.
8. Latenz-tolerant + kostenkritisch + viel Volumen → **Message Batches** (−50 %).
9. Kontextproblem → prunen, compacten, Subagent isolieren, Just-in-time-Retrieval – **nicht** „größeres Fenster".
10. Bei Fehlern: 4xx **nicht** retryen; 429/5xx/529 mit Backoff + Jitter.

Typische Distraktoren
„Größeres/stärkeres Modell", „Temperature erhöhen/auf 0", „GROSSBUCHSTABEN im Prompt", „Iterations-Limit als primärer Stopp", „400er retryen", „Multi-Agent als Default", „bessere Modelle widerstehen Injection".

D1 · Agents & Workflows 14,7 %

Workflow vs. Agent
Workflow: LLM-Calls + Tools über **vordefinierte Codepfade** – dein Code entscheidet. **Agent**: das Modell steuert dynamisch eigene Schritte und Tool-Nutzung in einer Schleife mit Feedback aus der Umgebung.

Agent ja/nein? Complexity · Value · Viability · Cost of error. Feste Abfolge → Workflow. Offene, unvorhersehbare Schritte → Agent.

Workflow-Patterns (Signal in der Frage)

Prompt Chaining	feste sequentielle Schritte, Gates dazwischen
Routing	klar getrennte Input-Kategorien; billiges vs. teures Modell
Parallel – Sectioning	*verschiedene* unabhängige Teilaufgaben gleichzeitig (z. B. Antwort + Guardrail)
Parallel – Voting	*dieselbe* Aufgabe N-mal → Konsens
Orchestrator-Workers	Teilaufgaben **vorher unbekannt**, zur Laufzeit bestimmt
Evaluator-Optimizer	klare Kriterien, iterative Verbesserung
Agent	offen, unvorhersehbar, Umgebungs-Feedback

Subagents

+ Kontext-Isolation, Parallelität, Spezialisierung, billigere Modelle für Teilaufgaben. **−** mehr Tokens, Koordinationsaufwand.

Falle
Subagents starten mit **frischem Kontext** – sie teilen kein Gedächtnis mit dem Parent. Fehlt Info → explizit im Delegations-Prompt mitgeben (nicht ganzes Transkript, nicht Modell wechseln). Nur das Endergebnis kommt zurück.

Agent-Loop (manuell)

```
while True:
  r = client.messages.create(
        model=…, max_tokens=…, system=…,
        tools=tools, messages=msgs)
  msgs.append({"role": "assistant",
               "content": r.content})
  if r.stop_reason != "tool_use":
      break
  results = [run(b) for b in r.content
             if b.type == "tool_use"]
  # ALLE tool_results in EINER User-Message
  msgs.append({"role": "user",
               "content": results})
```

Fallen
Loop-Ende per `stop_reason`, nicht „Antwort enthält Text". Tool-Results nie in getrennten User-Messages. Iterations-Cap = Sicherheitsnetz, nicht Hauptabbruch.

Bau-Optionen

	Loop	Hosting	Built-in Tools
Manueller Loop	du	du	keine
Tool Runner	SDK-Helper	du	keine (nur deine Tools)
Agent SDK	Claude-Code-Harness	**du**	Read, Write, Edit, Bash, Glob, Grep, Web; Subagents, MCP, Hooks, Sessions
Managed Agents (beta)	Anthropic	**Anthropic** (Sandbox pro Session)	Bash, Files, Code Exec, Skills, MCP, Vaults, versionierte Agents

Tool Runner = `client.beta.messages.tool_runner` im `anthropic`-SDK.

Agent SDK (`claude-agent-sdk`): `query()` (one-off) vs. `ClaudeSDKClient` (multi-turn) · `allowed_tools` = **auto-approve** (keine Einschränkung!) · `disallowed_tools` = verbieten · `permission_mode` · `can_use_tool` Callback · `setting_sources` · `mcp_servers` · `@tool` + `create_sdk_mcp_server` → Name `mcp__server__tool` · `hooks={"PreToolUse": [HookMatcher(...)]}` · `resume` / `fork_session` · `max_turns`, `max_budget_usd` · `ResultMessage.total_cost_usd`.

Memory ist nichts, was das Modell „hat": etwas muss in einen Store schreiben und wieder in den Kontext laden (Memory-Tool: *deine* App implementiert die Speicheroperationen). Framework (LangGraph etc.) nicht für Ein-Call-Probleme.

Harte Geschäftsregeln
„Ab $10.000 immer Freigabe" → Workflow-Gate/Hook im Code, nicht System-Prompt.

D2 · Applications & Integration

2.3 Messages API

```
POST https://api.anthropic.com/v1/messages
Header: x-api-key
        anthropic-version: 2023-06-01
        content-type: application/json
        [anthropic-beta]
Pflicht: model, max_tokens, messages
         (user zuerst, alternierend)
Optional: system, tools, tool_choice, stream,
  thinking, output_config{effort,format},
  stop_sequences, metadata{user_id}
```

- API ist **stateless** → gesamte History jedes Mal mitsenden. System-Prompt = Top-Level-Feld `system`.
- **Prefill** auf aktuellen Modellen nicht unterstützt → Structured Outputs. **Sampling-Params** (temperature …) auf neuesten Modellen abgelehnt.
- Weitere Endpoints: `/v1/messages/count_tokens`, `/v1/messages/batches`, `/v1/files`, `/v1/models`.

`stop_reason`	Reaktion
`end_turn`	fertig
`tool_use`	Tools ausführen → eine User-Msg mit allen `tool_result`s → erneut aufrufen
`max_tokens`	abgeschnitten → Limit erhöhen/fortsetzen, **nicht** als vollständig parsen
`stop_sequence`	eigene Stop-Sequenz getroffen
`pause_turn`	Server-Tool-Loop pausiert → erneut senden
`refusal`	Sicherheits-Ablehnung, **HTTP 200** → keine Tools ausführen, Fallback
`model_context_window_exceeded`	Fenster voll → Input reduzieren

Streaming (SSE – nicht WebSockets!)

`message_start` → `content_block_start` → `content_block_delta` (`text_delta` · `input_json_delta` · `thinking_delta`) → `content_block_stop` → `message_delta` (stop_reason, usage) → `message_stop`. `ping` jederzeit; `error` auch mitten im Stream. **Gleiche Kosten**, besseres TTFT (wahrgenommene Latenz), vermeidet Timeouts bei langen Antworten.

Message Batches

−50 % auf alles · ≤ **100.000** Requests / **256 MB** · meist < 1 h, max **24 h** · Ergebnisse in **beliebiger Reihenfolge** → **per `custom_id` zuordnen** · 29 Tage abrufbar · Status `succeeded / errored / canceled / expired`. Ideal: Nacht-Jobs, Evals, Massenklassifikation – nicht für Echtzeit.

Thinking & Effort

- Aktuelle Modelle: `thinking: {"type":"adaptive"}` + `output_config.effort: low · medium · high · xhigh · max`.
- Legacy (ältere Modelle, z. B. Haiku 4.5): `{"type":"enabled","budget_tokens":N}`, N ≥ 1024 und < max_tokens.
- Thinking-Blöcke im Tool-Loop **unverändert zurückgeben**. Thinking wird als **Output** abgerechnet – auch wenn Anzeige `omitted` / `summarized`.
- **Effort** = Hauptregler Qualität ↔ Kosten/Latenz (weniger Effort = weniger, konsolidierte Tool-Calls, knappere Antworten). **Fast Mode** (Opus): schneller, Premium-Preis.

Multimodal, Files, Citations

Bilder (base64/URL/`file_id`) **vor** die Frage; ≈ [w/28]·[h/28] Tokens; zu große werden verkleinert; ≤ 600/Request. **PDFs** = Text + Seitenbilder. **Files API**: einmal hochladen → `file_id` wiederverwenden. **Citations**: `cited_text` + Position; inkompatibel mit Structured Outputs.

Plattformen

Claude API (neue Features zuerst) · **Bedrock** (IAM, `anthropic.`-Prefix) · **Vertex AI** (GCP ADC, Projekt + Region) · **Foundry** (Azure). Gleiche Message-Struktur, aber **keine Feature-Parität** → pro Plattform prüfen.

2.5 App Design (8,6 % – größter Einzel-Skill)

> **Principal Hierarchy**
> Anthropic-Policies → **Operator** (System-Prompt/Entwickler) → **User** (Human-Turn). Inhalte aus Tool-Results, Dokumenten, Webseiten, Dateien = **Daten, keine Anweisungen**.

- Bare API ≠ claude.ai: kein Produkt-System-Prompt, kein Memory, keine Tools – alles muss der Operator definieren.
- Abgerufene Inhalte **nicht** in den System-Prompt kopieren → User-Turn, in Tags abgegrenzt.
- Schemas: nicht jedes Feld `required` → `null` / `unknown` erlauben, sonst erfindet das Modell Werte.
- API-Key nie im Browser/Mobile-Client → Backend-Proxy mit User-Auth und Limits.
- Side-Effects → Approval-Gates; Aktionen mit dem Scope des **Users** autorisieren.

2.4 Software Engineering Foundations (7,4 %)

- **Idempotenz**: Retry nach Timeout kann Side-Effect duplizieren (Zahlung, Mail) → Idempotency Keys / Status prüfen.
- **Concurrency begrenzen**: kein ungebremstes `asyncio.gather` über 10.000 Calls → Semaphore, Backoff auf 429, oder Batches API.
- **Einen SDK-Client wiederverwenden** (Connection-Reuse), nicht pro Request neu erzeugen.
- Timeouts: Python `timeout=10` = 10 **s**; TypeScript `timeout: 10` = 10 **ms**.
- Kleine, testgestützte, reviewbare PRs statt Mega-Refactor. **Unabhängiges Review** (andere Session, Subagent, Mensch) statt Selbst-Review im selben Kontext.
- Strukturiertes Logging inkl. Request-IDs, Usage, Stop-Reasons; Secrets aus Env/Secret Store.
- Transport: Output zum Browser → **SSE**; bidirektional/kontinuierlich (Voice, Co-Editing) → WebSockets; lange async Jobs → Polling mit Backoff oder **Webhook**.

2.1 Requirements (3,4 %)

> **Erfolgskriterien zuerst**
> Spezifisch, messbar, erreichbar, relevant – **vor** dem Prompt-Engineering definieren, dann Evals bauen. Mehrdimensional: Task-Fidelity, Konsistenz, Relevanz, Ton, Latenz, Kosten, Privacy.

- „Team tweakt Prompts, weiß nicht ob's besser wird" → Kriterien + Eval-Set, kein neuer Prompt-Trick.
- **Latenz**: TTFT ↔ Streaming; lange statische Präfixe ↔ Caching; kürzerer Output ↔ schnellere Gesamtzeit; erst dann kleineres Modell.
- **Durchsatz** gegen Rate Limits prüfen (RPM, ITPM, OTPM) → Batch, Queue + Backoff, Caching (Cache-Reads zählen meist nicht gegen ITPM), höherer Tier.

2.2 Systems Life Cycle (2,8 %)

LLM-Systeme sind empirisch → **iterativ/agil mit automatisierten Eval-Gates**, nicht Wasserfall.

> **Modell-Migration (Checkliste)**
> 1. Release Notes/Migration Guide lesen · 2. neue Model-ID im Branch pinnen, Params anpassen · 3. **volle Eval-Suite** (Qualität, Latenz, Tokens, Kosten vs. Baseline) · 4. Prompts nachjustieren (neuere Modelle wörtlicher → Übertreibungen entfernen) · 5. Shadow/Canary, Stop-Reasons/Refusals/Fehler monitoren · 6. Feature-Flag-Umschaltung, Rollback bereit bis Retirement.

Nicht bis zum Retirement-Datum warten · Benchmark-Sieg ≠ besser für *deinen* Task.

2.6 Configuration Management (4,1 %)

> **Guidance vs. Enforcement**
> CLAUDE.md „nie `migrations/` editieren" = wahrscheinlich befolgt. `permissions.deny: ["Edit(./migrations/**)"]` = **erzwungen**.

- Permission-Listen werden über Scopes **gemergt**; **deny schlägt allow**, egal wo definiert.
- Persönliches → `.claude/settings.local.json` / `CLAUDE.local.md` (gitignored). Secrets → Env/Secret Store.
- Produktion: Model-ID **pinnen** (keine wandernden Aliasse), loggen, bewusst upgraden. Pro Umgebung eigene Keys/Workspaces.

Muster: Request mit Caching, Effort & Fehlerbehandlung

```
client = anthropic.Anthropic()  # 1x, reuse
try:
  r = client.messages.create(
    model="claude-sonnet-…", max_tokens=1024,
    system=[{"type": "text",
             "text": STATISCHE_ANWEISUNG,
             # Breakpoint ans Ende des
             # gemeinsamen Präfixes:
             "cache_control":
               {"type": "ephemeral"}}],
    # Daten in den User-Turn, mit Tags:
    messages=[{"role": "user", "content":
      f"<doc>{doc}</doc>\n{frage}"}],
    thinking={"type": "adaptive"},
    output_config={"effort": "medium"})
except anthropic.RateLimitError:
  ...  # 429: Backoff, retry-after
except anthropic.APIStatusError as e:
  ...  # 4xx fixen, 5xx/529 retry
if r.stop_reason == "max_tokens":
  ...  # abgeschnitten!
text = "".join(b.text for b in r.content
               if b.type == "text")
r.usage.cache_read_input_tokens  # Hit?
```

> **Offizielle Beispielfrage – Muster**
> 10.000 Dokumente über Nacht, nicht dringend, Kosten im Fokus → **Message Batches API** (nicht: parallele synchrone Calls, `max_tokens` senken, kleinstes Modell). Richtige Antwort = passender **Mechanismus** für die eigentliche Anforderung.

D8 · Tools & MCPs 10,6 %

8.1 Tool-Implementierung

- Definition: `name`, **`description`** (Zweck, *wann* nutzen, Inputs, Outputs, Grenzen – wichtigster Hebel!), `input_schema` (JSON Schema), optional `strict: true` (+ `additionalProperties: false`), `input_examples`.
- Results: **eine** User-Message, jede `tool_use_id` beantwortet; Fehler als `is_error: true` mit **handlungsleitender** Meldung (nicht Exception aus dem Loop werfen).
- Wenige, aufgabenorientierte Tools statt 60 Endpoint-Wrapper. Große Kataloge → **Tool Search** (`defer_loading`); viele verkettete Calls → **Programmatic Tool Calling**.
- Schreibende/irreversible Aktionen → Bestätigung, egal wie gut das Modell ist.

`tool_choice`	
`auto`	Default, Modell entscheidet
`any` / `tool`	erzwingen (auf neuesten Modellen abgelehnt → auto + Prompt / strict / Structured Outputs)
`none`	keine Tools
`disable_parallel_tool_use`	≤ 1 Call pro Turn

Client Tools (du führst aus): Custom, bash, text editor, memory, computer use. **Server Tools** (Anthropic führt aus): web search, web fetch, code execution, tool search, advisor.

8.2 MCP-Server

Host → Client (1 pro Server) → Server · **JSON-RPC 2.0**. Lifecycle: `initialize` → Capabilities → `notifications/initialized` → `tools/list`, `tools/call`, `resources/read`, `prompts/get`; `list_changed`-Notifications.

Primitive	gesteuert von	Zweck
Tools	Modell	Aktionen
Resources	Anwendung	read-only Kontext (URIs, Templates) – z. B. Schemas, Docs
Prompts	User	Templates / Slash-Commands

- **Transports: stdio** = lokaler Subprozess (**stdout = Protokoll** → **Logs nach stderr**, kein `print()`!) · **Streamable HTTP** = remote, mehrere User, OAuth · HTTP+SSE = legacy.
- Client-Features: Elicitation, Roots (Sampling im neuesten Spec deprecated).
- SDKs: Python `MCPServer` / `FastMCP` + `@mcp.tool()`; TS `McpServer.registerTool`. Testen mit **MCP Inspector**.
- Claude Code: `claude mcp add --transport stdio|http`, Scopes local / project (`.mcp.json`) / user; Tool-Namen `mcp__server__tool`.
- API: **MCP Connector** (`mcp_servers` + `mcp_toolset`) – nur Remote-Server.

8.3 Agentic Customization – Entscheidungstabelle

> **Tools geben Fähigkeiten, Skills geben Know-how**
> Tool/MCP: „Was kann Claude tun/erreichen?" Skill: „Wie macht Claude diese Art Aufgabe gut?" Kann es → macht es aber inkonsistent → Skill.

Bedarf	Lösung
Generische Fähigkeit (Web, Code Exec, Dateien, Memory)	**Built-in Tool**
App-spezifische Aktion/Daten in einer App	**Custom Tool**
Wiederverwendbare Integration, unabhängig gewartet, mehrere Hosts	**MCP-Server**
Prozedur / Expertise / Templates, bei Bedarf geladen	**Skill**
Immer gültige Projektfakten	**CLAUDE.md** / rules
Isolation, Parallelität, Spezialist	**Subagent**
Muss immer / darf nie	**Hook** / Permission-Regel
Bündel verteilen	**Plugin**

D3 · Claude Code 3,1 %

CLAUDE.md-Hierarchie: managed → `~/.claude/CLAUDE.md` → `./CLAUDE.md` bzw. `./.claude/CLAUDE.md` → `./CLAUDE.local.md`. Vorfahren beim Start, Unterordner on demand, alles **konkateniert**. `@imports` (≤ 4 Ebenen). `.claude/rules/*.md` mit `paths:`. Ziel < 200 Zeilen. = **Kontext, keine Durchsetzung**.

settings.json-Priorität: managed → CLI `--settings` → `.claude/settings.local.json` → `.claude/settings.json` → `~/.claude/settings.json`. Listen mergen. Auswertung **deny** → **ask** → **allow**.

Permission Mode	
`default`	fragt bei erster Nutzung
`acceptEdits`	Edits + übliche FS-Befehle automatisch
`plan`	nur lesen/planen
`auto`	Classifier prüft statt Prompts
`dontAsk`	alles, was fragen würde, wird abgelehnt
`bypassPermissions`	keine Prompts – **nur isolierte Sandbox/VM**

Regel-Syntax: `Bash(npm run test *)` · `Read(./.env)` · `Edit(src/**)` · `WebFetch(domain:x.com)` · `mcp__github__*`

Bausteine: Skills (`SKILL.md`, on demand; `disable-model-invocation`, `allowed-tools`, `context: fork`, `$ARGUMENTS`, `!`cmd``) · Commands (`.claude/commands/*.md` = Skills) · **Subagents** (`.claude/agents/*.md`: name, description, tools, model; eigener Kontext) · Hooks · MCP (`.mcp.json`) · **Plugins** (`.claude-plugin/plugin.json`, `/plugin:skill`; vor Installation prüfen – Hooks/MCP laufen als du).

Commands: `/init /memory /clear /compact [focus] /context /rewind /resume /model /effort /config /permissions /hooks /agents /mcp /plugin /review /security-review /doctor`

Headless/CI: `claude -p "…" --output-format text|json|stream-json --allowedTools … --permission-mode … --max-turns N --json-schema … --resume <id> --bare`. Ohne `--allowedTools`/passenden Mode werden prompt-pflichtige Aktionen in CI **abgelehnt**.

Sessions: `-c`/`--continue`, `-r`/`--resume`, `--fork-session`. Zwischen Aufgaben `/clear`; Exploration an Subagents delegieren; wiederkehrende Prozeduren → Skill statt CLAUDE.md.

D7.3 · Hooks 1,0 %

Events: `PreToolUse` (blocken/ändern) · `PermissionRequest` · `PostToolUse` (formatieren/linten/auditieren – läuft **nach** Ausführung, kann nicht verhindern!) · `UserPromptSubmit` · `Stop` / `SubagentStop` (Beenden blocken) · `SessionStart` · `PreCompact` · `Notification` · `SessionEnd`.

Config: Event → `matcher` (Tool-Name/Regex) → Handler (`command`, `http`, `mcp_tool`, `prompt`, `agent`). Input als JSON auf **stdin**.

Exit **0**	ok (optional JSON auf stdout)
Exit **2**	**blockieren** – stderr geht an Claude
anderer (z. B. 1)	nicht-blockierender Fehler – Tool läuft trotzdem!

JSON: `hookSpecificOutput.permissionDecision: allow|deny|ask` (+ Reason), `updatedInput`, `additionalContext`; `decision: "block"` (Stop); `continue: false` + `stopReason`.

```
{"hooks": {"PreToolUse": [{
   "matcher": "Bash",
   "hooks": [{"type": "command",
              "command": "./guard.sh"}]
}]}}
```

Muster: `.claude/settings.json`

```
{"permissions": {
   "allow": ["Bash(npm run test *)",
             "Edit(src/**)"],
   "ask":   ["Bash(git push *)"],
   "deny":  ["Read(./.env)",
             "Edit(./migrations/**)"]}}
```

Muster: Skill `.claude/skills/release/SKILL.md`

```
---
name: release
description: Erstellt Release-Notes und
  Tag. Nutzen, wenn ein Release
  vorbereitet werden soll.
allowed-tools: Bash(git *), Read
disable-model-invocation: true  # nur /release
---
1. Änderungen seit letztem Tag sammeln …
```

Nur `name` + `description` liegen dauerhaft im Kontext (Progressive Disclosure); der Body lädt erst bei Bedarf.

Muster: MCP-Server (Python, stdio)

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("tickets")

@mcp.tool()
def get_ticket(ticket_id: str) -> dict:
    """Lädt ein Ticket per ID. Nutzen bei
    Fragen zu Status/Details."""
    ...  # Logs → stderr, nie print()

@mcp.resource("schema://tickets")
def schema() -> str:
    ...  # read-only Kontext

if __name__ == "__main__":
    mcp.run()  # Transport: stdio
```

> **Offizielle Beispielfrage – Muster**
> Interner Inventar-REST-Service für mehrere Claude-Apps, unabhängig wartbar → **MCP-Server** (nicht: Logik in System-Prompts, Daten in jeden Request, Built-in Tool).

D5 · Model Selection & Optimization

5.1 LLM-Grundlagen

- Token-basiert; Tokenizer unterscheiden sich je Modellgeneration → `count_tokens`-Endpoint / `usage` nutzen, keinen generischen Tokenizer.
- **Kontextfenster** umfasst Input **und** Output (inkl. Thinking).
- Nicht-deterministisch: niedrige Temperature ≠ garantiert identisch (und neueste Modelle lehnen Sampling-Params ab) → Determinismus über Structured Outputs, Code, Caching.
- Thinking: besser bei Mathe, Code, mehrstufigem Tool-Use – kostet Output-Tokens und Latenz.

5.3 Modellwahl

Haiku	hohes Volumen, niedrige Latenz, Klassifikation, Routing, Sub-Agents
Sonnet	ausgewogener Default für Produktion & Coding
Opus	komplexes Reasoning, lange Agenten-Läufe, Orchestrator
Fable (Top)	anspruchsvollste Arbeit, Qualität ≫ Kosten

Kriterium: **Kosten pro erfolgreich erledigter Aufgabe** auf **deinen** Evals. Weder „immer das stärkste" (Verschwendung, Latenz) noch „immer das billigste" (mehr Turns/Retries). Thinking-Konfig variiert je Generation (adaptive ab 4.6, budget älter).

5.4 Kosten, Tokens & Prompt Caching

Token-Typ	× Basis-Input
Cache-Write 5 Min	**1,25×**
Cache-Write 1 h	**2×**
Cache-Read	**~0,1×**
Batch	**0,5×** auf alles (kombinierbar)
Output	≈ **5×** Input-Preis

- Präfix-Reihenfolge **tools** → **system** → **messages**; jede Byte-Änderung invalidiert alles danach. Max. **4 Breakpoints**; modellabhängige Mindestlänge; TTL wird bei Hit erneuert; pro Workspace & Modell.
- Break-even: 5-Min-TTL ab dem **2.** Request, 1-h-TTL ab dem **3.**
- **Cache-Killer:** Timestamp/IDs früh im System-Prompt, unsortiertes JSON, wechselnde Tools, Modellwechsel, Thinking/Effort-Toggles, History-Edits, Breakpoint hinter dem variablen Teil.
- Caching senkt nur **Input**-Kosten, nicht Output. `input_tokens` = nur ungecachter Rest → Gesamt = `input + cache_creation + cache_read`.
- **Hebel-Reihenfolge:** Caching · Trimmen · Output-Hygiene · Batches → Effort · Modell-Tier · Routing.

5.2 Technische Grundlagen (6,1 %)

Status	Retry?
400 invalid_request · 401 auth · 402 billing · 403 permission · 404 not_found · 413 too large	**Nein** – fixen
429 rate_limit (`retry-after` beachten) · 500 api_error · **529 overloaded** · Netzwerk	**Ja** – exp. Backoff + Jitter

SDKs retryen 408/409/429/5xx/Connection standardmäßig **2×**. Typisierte Exceptions spezifisch→allgemein fangen. Request-IDs loggen. Bei Overload Fallback-Modell oder Queue.

D6 · Prompt & Context Engineering

6.2 Prompting

- Klar & spezifisch (wer/was/warum/Format/Länge) · Kontext/Motivation statt CAPS · **Rolle im System**, **Daten im User-Turn** in XML-Tags.
- 3–5 diverse Beispiele in `<example>` · lange Dokumente **oben**, Frage **unten** · erst relevante Zitate extrahieren lassen.
- Step-by-step bzw. adaptive Thinking · Prompt Chaining für komplexe Aufgaben.
- Neuere Modelle wörtlicher → „IMPORTANT/ALWAYS" weglassen (Over-Triggering). Eine Änderung pro Iteration + Evals.
- Inputs sanitizen: Länge, Steuer-/Zero-Width-Zeichen, Delimiter escapen. Per-Request-Daten nie in den System-Prompt (Caching, Trust).

6.1 Context Engineering

Leitprinzip
Kontext ist endlich, abnehmender Grenznutzen („context rot"). Ziel: **kleinste Menge hochrelevanter Tokens**.

Techniken: **Just-in-time Retrieval** (Referenzen statt Volltext) · **Compaction** · **Tool-Output prunen** (Tool fixen statt compacten) · strukturierte Notizen/externe Memory · **Subagent-Isolation** mit fokussiertem Auftrag · bei serverseitiger Compaction komplettes `response.content` zurückgeben.

6.3 Output Handling

- Parser-abhängig → **Structured Outputs** (`output_config.format`) oder `strict` Tool-Schema – nicht „bitte JSON".
- Structured Outputs garantieren **Form, nicht Wahrheit** → Business-Validierung, Grounding.
- Blöcke nach `type` auswählen (thinking, tool_use, mehrere text) – nicht `content[0]`.
- Selbstberichtete „confidence: 0.98" ist kein kalibrierter Wert. Output nie als rohes HTML rendern → encoden/sanitizen.

D7 · Security & Safety 8,1 %

Lethal Trifecta
(1) Zugriff auf private Daten + (2) untrusted Content + (3) externe Kommunikation/folgenreiche Aktionen → **ein Bein entfernen**.

- Prompt Injection: direkt & **indirekt** (über Docs/Web/Tool-/MCP-Output). Gegenmittel: Isolation/Delimiting, Least Privilege, Approvals, deterministische Gates, Output-Encoding – **Defense in Depth**, nie eine einzelne Maßnahme.
- Access Control in Code & Datenzugriff der Tools; RAG-ACLs **beim Retrieval**. PII minimieren/redigieren.
- **Fail closed** auf Hochrisiko-Pfaden (Classifier-Timeout ≠ allow).
- Keine Admin-Service-Accounts für Agents → **delegierte, user-scoped Credentials**; OAuth für Remote-MCP.
- **Secrets** nie in Prompts, Clients oder Repos → Secret Manager; pro Umgebung eigene Keys/Workspaces mit Spend-Limits; Usage & Audit-Logs monitoren.
- Key geleakt → **sofort widerrufen, dann rotieren** (nicht Git-History umschreiben).

D4 · Eval, Testing & Debugging 2,6 %

Isolations-Prozedur
1. Exakten Request + Raw-Response erfassen · 2. direkt replayen (curl/Console) → Antwort korrekt? → Bug **downstream** (Parsing, State, Rendering) · 3. Request prüfen (fehlende History, abgeschnittene Docs, falsche Prompt-Version, Tool-Results fehlen, falsche Model-ID) = immer noch Integrations-Bug · 4. Erst dann Modellverhalten → Prompt/Tools/Kontext/Effort/Modell + Fall ins Eval-Set · 5. mehrfach reproduzieren.

Evals: code-graded (exakt, schnell) > LLM-as-Judge (klare Rubrik) > Human (teuer, für Kalibrierung). Repräsentative + Edge-Cases, Regressionstests bei jeder Änderung, Production-Monitoring (Stop-Reasons, Refusals, Fehlerraten, Kosten), Trace-Analyse bei Agenten.

Häufige Verwechslungen

Tool Runner ↔ Agent SDK	nur deine Tools ↔ volle Claude-Code-Harness
Sectioning ↔ Voting	verschiedene Teilaufgaben ↔ dieselbe Aufgabe N-mal
PreToolUse ↔ PostToolUse	verhindern ↔ nachbearbeiten/auditieren
Hook Exit 2 ↔ Exit 1	blockiert ↔ nur Fehler, Tool läuft
`allowed_tools` ↔ `disallowed_tools`	auto-approve ↔ verbieten
CLAUDE.md ↔ `permissions.deny`	Guidance ↔ Enforcement
Skill ↔ CLAUDE.md	on demand, Prozedur ↔ immer geladen, Fakten
MCP Tool ↔ Resource ↔ Prompt	Modell ↔ Anwendung ↔ User steuert
Streaming ↔ Batch	gleiche Kosten, besseres TTFT ↔ −50 %, asynchron ≤ 24 h

Offizielle Beispielfrage – Muster
Webseite mit verstecktem Text „ignoriere vorherige Anweisungen …" → **Inhalt als untrusted behandeln, von Anweisungen trennen, Guardrails/Hooks** gegen sensible Aktionen. Nicht: Temperature erhöhen, User im System-Prompt bitten, größeres Modell.

Prüfungstaktik

- **Zeit:** ≈ 2:15 Min/Item. Nach ~60 Min solltest du bei ~27 sein. Schwere Items markieren (flag) und weiter – am Ende zurückkommen.
- **Stamm zuerst** nach der eigentlichen Anforderung durchsuchen: Kosten? Latenz (TTFT vs. gesamt)? Sicherheit? Wiederverwendung? „muss immer"?
- Antworten eliminieren, die **nicht durchsetzbar** (nur Prompt), **überdimensioniert** (Multi-Agent, größtes Modell) oder **am Problem vorbei** sind.
- Bei Multiple Response **exakt** die geforderte Anzahl wählen. Keine Frage leer lassen.
- Schwerpunkte fürs Lernen: **D2 (33 %)** + **D5 (17 %)** + D1 (15 %) = ⅔ der Punkte.
- **Bereit**, wenn du in vollen Mock-Exams stabil **≥ 80 %** erreichst (≈ 70 % ≙ 720).